03 iDeliveryFood Prompts Apps Portais

Regras desta revisão — aplicação integral

Este documento conserva o conteúdo e os módulos do PDF anexado, com alterações nas instruções afetadas, sem resumo. As regras abaixo se aplicam a todos os módulos e critérios de aceite deste arquivo.

Mobile nativo e testes por plataforma

Android: Kotlin e Jetpack Compose; iOS: Swift e SwiftUI, projetos separados. Sem app híbrido/React. Implementar e testar cada plataforma independentemente: unitários de domínio e ViewModel, integração API, UI, deep links, sessão, permissões, offline e acessibilidade. CI Android com toolchain apropriado; CI iOS em runner macOS com Xcode. Assinatura e contas de distribuição são externas ao Git. Falta de ambiente Apple deve aparecer como pendência, nunca como build/teste concluído. Web/PWA permanece entrega web separada, jamais substitui os apps nativos.

MCP exato fornecido pelo proprietário

{
"mcpServers": {
"supabase": {
"serverUrl": "https://mcp.supabase.com/mcp?project_ref=dwsoochnfxldfvcaijie&features=docs%2Caccount%2Cdatabase%2Cdebugging%2Cdevelopment%2Cfunctions%2Cbranching%2Cfeedback"
}
}
}

SQL remoto e seed

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

GitHub depois dos testes

O Antigravity deve executar commit e push em https://github.com/JuanBMMontesinos/iDeliveryFood.git depois de testar o código e revisar diff e segredos. Verificar branch/remotes e preservar histórico; não

usar force-push, mudar visibilidade ou contornar proteção. Respeitar gates de aprovação do ambiente e autenticar por mecanismo seguro. Executar testes locais antes do push e acompanhar CI remoto depois; falha de CI requer correção e novo push, sem alegar release aprovado. Confirmar SHA remoto. Push não é deploy nem publicação nas lojas.

Idioma principal

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

Documentação e segurança preservadas

Manter documentação completa, testes unitários, integração, RLS/pgTAP, e2e e controles de cybersegurança dos módulos originais. Atualizar arquitetura, ERD, migrations aplicadas, seed, API, matriz de permissões, instruções nativas de build/teste, i18n, CI/CD, operação e evidências. Não incluir segredos; não declarar certificação de segurança sem validação independente.

Conteúdo integral revisado do documento anexado

iDeliveryFood — Apps e portais separados

Como usar

PACOTE REVISADO — PROMPTS POR APLICATIVO, PORTAL E MÓDULO

Este material substitui o pacote anterior. É uma especificação e conjunto de comandos para o agente construir o iDeliveryFood incrementalmente; não é código já implementado nem certificação de produção.

Os prompts cobrem explicitamente app web e app mobile do cliente, app mobile do entregador, portal do parceiro, gestor de pedidos, portal de redes, console logístico, área de frota, administração central e backoffices financeiro, atendimento, comercial, marketing, relatórios, risco e privacidade, além da infraestrutura comum.

Como usar: cole as instruções permanentes antes do prompt do módulo. Em uma nova sessão, repita-as e peça ao agente para ler docs/DELIVERY_STATUS.md e os contratos existentes. Execute um módulo por tarefa, respeitando dependências. Não envie todos para execução simultânea. IDs indicam módulos, não obrigatoriamente a ordem numérica.

Projeto Supabase corrigido: dwsoochnfxldfvcaijie. Executar migrations e seed no ambiente de desenvolvimento autorizado conforme esta revisão. Rotacionar a senha exposta; arquivos não contêm senha nem URI credenciada.

Não há publicação GitHub, acesso ao banco ou implantação realizada por este pacote. Contas externas, contratação de serviços, credenciais seguras, revisão jurídica e aprovações permanecem responsabilidade do proprietário.

Instruções permanentes

INSTRUÇÕES PERMANENTES — COPIAR COMO CONTEXTO EM TODAS AS SESSÕES

Você é a equipe de engenharia do iDeliveryFood dentro do Antigravity. Execute uma etapa por vez e produza código funcional, não apenas planos, protótipos ou descrições. Inspecione o repositório e siga os contratos/ADRs existentes. Não substitua stack ou reescreva módulos prontos sem justificar e obter aprovação. Identifique esta entrega pelo ID do módulo, mantenha docs/DELIVERY_STATUS.md atualizado e não afirme conclusão sem evidências.

REPOSITÓRIO: alvo solicitado pelo proprietário:

https://github.com/JuanBMMontesinos/iDeliveryFood.git. Verifique primeiro acesso, conteúdo, branch, remotes e estado local. Não suponha que exista ou esteja vazio. Após testes aprovados, executar commit e push ao repositório solicitado e aplicar migrations no Supabase autorizado. Respeitar aprovações do ambiente; não alterar visibilidade, usar force-push ou executar operações destrutivas sem autorização específica. Antes de qualquer

publicação: revisar diff, testes e secret scan; não expor credenciais ou dados pessoais.

SUPABASE — PROJETO CORRIGIDO E EXECUÇÃO REMOTA: projeto autorizado dwsoochnfxldfvcaijie. Configure o MCP com o JSON exato desta revisão e autenticação segura. Inspecione o schema, crie/altere tabelas, índices, constraints, funções, RLS, grants, Storage e demais recursos necessários; execute os SQLs no Supabase por migrations versionadas e verifique o resultado real. Não entregue somente

arquivos SQL sem aplicá-los. A senha anteriormente exposta deve ser rotacionada; nunca reproduzi-la ou versioná-la. Não gravar tokens no Git. Identifique o ambiente antes de inserir seed: usar desenvolvimento/teste. Em produção ou ambiente desconhecido, confirmar antes de gravar dados sintéticos. Não executar DROP/TRUNCATE/reset remoto ou alterações destrutivas sem aprovação específica. Registrar migration, resultado, validação e plano de recuperação sem segredos.

DADOS FICTÍCIOS PERSISTIDOS — SEED OBRIGATÓRIO: criar supabase/seed.sql com dados sintéticos coerentes para desenvolvimento/teste e executá-lo no Supabase autorizado. Tudo que representa dados de negócio deve persistir no Supabase: lojas, produtos, adicionais, preços, horários, pedidos, eventos e métricas derivadas. NUNCA colocar dados chumbados no HTML/páginas, componentes, classes dos apps, arrays locais ou respostas falsas de API. UI consulta banco/API com RLS; vazio real continua sendo estado válido. Seed idempotente com IDs estáveis, relações válidas e marcação de dados de teste; executá-lo duas vezes em teste sem duplicação. Não usar dados pessoais reais nem senhas no seed. Identidades de teste via Auth suportado, segredos fora do Git. Seed não dispara cobranças ou mensagens externas. Dados fictícios não entram em produção. Doubles/fixtures de unitários são permitidos só nos testes isolados, nunca importados pelo runtime. Traduções e configuração estática não são dados de negócio; cache apenas derivado do backend. Provedor ausente não pode retornar sucesso fictício.

ARQUITETURA DE REFERÊNCIA: monorepo com backend/web TypeScript e workspace pnpm para apps/customer-web (Next.js), apps/partner-web (Next.js), apps/admin-web (Next.js), apps/api (NestJS) e apps/worker. Mobile com código NATIVO separado: apps/customer-android e apps/courier-android em Kotlin/Jetpack Compose; apps/customer-ios e apps/courier-ios em Swift/SwiftUI. Se algum novo app mobile for habilitado, também terá implementações Android e iOS independentes. NÃO usar React Native, Expo, Flutter, Ionic, Capacitor ou solução híbrida; não encapsular páginas web como app. Compartilhar contratos OpenAPI e especificações de domínio, não UI/runtime TypeScript com os apps. packages/contracts, domain, authz, api-client, ui-web, config são para backend/web; clientes nativos próprios por plataforma. Gradle e Xcode gerenciam seus projetos. Validar versões estáveis, lockfiles e ADR de migração se já houver código. Logística e portais de rede/frota mantêm áreas web autorizadas próprias; um backend comum para todas as interfaces.

CONTRATOS: API versionada /v1 com OpenAPI e cliente tipado; Supabase Auth para identidade; PostgreSQL/Supabase como fonte de verdade; AuthZ no servidor e RLS/grants no banco para as superfícies acessíveis; Realtime somente em canais autorizados. Negócio e escrita privilegiada no backend; consultas diretas do cliente apenas quando explicitamente aprovadas e protegidas por RLS. Service-role e segredos exclusivamente no servidor, jamais bundles/public env. Roles não podem vir de metadata editável pelo usuário. Multi-tenant por organização e unidade; evitar caches compartilhados entre tenants. Finanças, auditoria e documentos sensíveis em superfícies restritas.
SEGURANÇA: MFA obrigatório para staff privilegiado; cookies/tokens apropriados a cada plataforma, validação server-side, CORS allowlist, CSP e headers na web, rate limits, proteção a CSRF/XSS/SQLi/SSRF/IDOR, uploads seguros, rotação/revogação, log redigido sem segredos/PII indevida. Mobile usa armazenamento seguro de tokens e permissões mínimas. Não armazenar dados de cartão. Endereço/localização/documentos com exposição e retenção mínimas. Mudança bancária, repasses e reembolsos acima da alçada exigem controle adicional. Threat model e checklist OWASP ASVS versionados como metas verificáveis, sem alegar certificação. Revisão jurídica e pentest externo continuam pendências de lançamento.
PADRÕES DE DOMÍNIO: centavos inteiros para BRL; datas persistidas UTC e exibidas America/Sao_Paulo; validação de timezone de horário de loja; UUIDs/IDs não substituem autorização; transações, constraints, índices e idempotency keys para mutações críticas; outbox e consumidores

idempotentes para tarefas assíncronas. Máquina de estados e histórico de eventos de pedido separados dos estados de pagamento e entrega. Não chamar provedor externo dentro de transação longa. Estado final do PSP exige validação de assinatura, correlação e reconciliação. Não confiar em preço, comissão, papel ou status enviado pelo cliente.

INTERFACE E i18n: inglês (en) como idioma principal e padrão inicial; usuário pode mudar para português brasileiro (pt-BR), com preferência persistida no Supabase para conta autenticada. Android usa recursos strings/plurals, iOS usa String Catalog/localization, web usa solução i18n com chaves, parâmetros, pluralização e fallback en. Não concatenar frases traduzíveis; traduzir validações, erros públicos, acessibilidade e notificações. Formatar datas/números/moeda por locale; BRL e região de operação não mudam só porque a interface está em inglês. Responsividade e acessibilidade; estados loading/vazio/erro/offline/sem permissão. Testar troca de idioma, persistência, fallback e paridade de chaves. O termo técnico é i18n (não i8n).

QUALIDADE: cada módulo traz testes unitários de regras, integração local com Supabase, casos de acesso negado e multi-tenant, contratos e e2e aplicáveis. CI exige lint, typecheck, testes, RLS/pgTAP, build e secret scan. Propor meta inicial de cobertura de 80% em código de domínio e cobertura de decisões críticas, sem manipular métricas; não é resultado garantido. Testes ausentes ou pulados explicitamente relatados. Nunca executar testes com cobrança ou dados sintéticos em produção.

FINAL DE CADA ETAPA: listar arquivos alterados; migrations; endpoints; telas; permissões; comandos executados e resultados reais; cobertura; evidências; riscos; pendências de configuração; atualização de docs e próximos passos. Se faltar credencial/aprovação, continuar apenas no trabalho seguro local e registrar BLOQUEADO, não CONCLUÍDO. Após os testes e secret scan, realizar commit e push no GitHub alvo, respeitando gates do ambiente e confirmando SHA remoto.

M05 — App web do cliente — portal de compra/PWA

PROMPT M05 — App web do cliente — portal de compra/PWA

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10; pagamento final depende M12.

OBJETIVO: Construir apps/customer-web completo, não só landing page.

IMPLEMENTAÇÃO E TELAS

Telas: entrada/cadastro/recuperação, localização e endereços, descoberta/busca/filtros, detalhe da loja, produto/adicionais/observação, carrinho, cotação e checkout, resultado/pendência de pagamento, acompanhamento e histórico do pedido, avaliação após entrega, perfil, privacidade e suporte. Carrinho de uma loja por pedido na primeira versão; troca pede confirmação. Totais vêm da cotação real. Mostrar frete, taxas e prazo somente se calculados. Revalidar indisponibilidade ao concluir. Checkout bloqueado se PSP ou política não configurados; não exibir pedido pago artificial. Acompanhamento por eventos autorizados com fallback de consulta. PWA com estratégia de cache que nunca vaza pedidos, endereços ou tokens; pedido/checkout sempre online e explícito. SEO só nas

páginas públicas aprovadas. Acessibilidade de teclado/leitor, formulários e erros.

DADOS, CONTRATOS E ACESSOS

Consumir api-client compartilhado; sem duplicar lógica de preços; /me/addresses, stores/catalog/quotes/orders/support.

TESTES OBRIGATÓRIOS

Componentes e formulário; e2e compra em ambiente de teste real/sandbox configurado; preço adulterado, produto indisponível, repetição de clique, sessão expirada e dados de outro cliente negados.

CRITÉRIOS DE ACEITE

Fluxo completo real quando configurado, com vazio/erro honesto caso contrário; documentação de telas e e2e.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M05.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M06 — App mobile do cliente — Android e iOS

PROMPT M06 — App mobile do cliente — Android e iOS

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10; M12 para pagamento; contratos de M05.

OBJETIVO: Construir apps/customer-android em Kotlin/Jetpack Compose e apps/customer-ios em Swift/SwiftUI, independentes e nativos, sem HTML embutido ou framework híbrido.

IMPLEMENTAÇÃO E TELAS

Android Kotlin/Jetpack Compose e iOS Swift/SwiftUI, implementações independentes com navegação nativa, cadastro/login/recuperação, seleção de endereço, permissão de localização opcional/manual, descoberta, loja/cardápio/adicionais, carrinho/cotação/checkout, pedidos/tracking, perfil/suporte e preferências de notificações. Tokens em armazenamento seguro; deep links allowlist para Auth/pagamento/pedido. Push apenas após consentimento e configuração real, revogável por dispositivo. Offline mostra dados marcados como desatualizados e não conclui compra. Não compartilhar token em URL/log. Configurar IDs, ícones/splash próprios sem imitar marca iFood. Builds separados Android/iOS com assinatura

providenciada fora do Git; publicação em lojas exige conta e aprovação do proprietário.

DADOS, CONTRATOS E ACESSOS

Contratos OpenAPI e regras especificadas em comum; clientes HTTP/SDK e domínio nativos próprios em Kotlin e Swift, sem runtime compartilhado; callback de pagamento validado no backend.

TESTES OBRIGATÓRIOS

Unitários/componentes, integração API, testes de navegação/deep links, token storage, permissão negada, retomada do app e e2e em dispositivo/emulador disponível.

CRITÉRIOS DE ACEITE

App executável Android/iOS conforme ferramentas disponíveis; resultados reais e instruções de build, assinatura e publicação sem alegar upload não realizado.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M06.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M07 — Portal web do parceiro — gestão do estabelecimento

PROMPT M07 — Portal web do parceiro — gestão do estabelecimento

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M04, M10; M13 para financeiro.

OBJETIVO: Construir área de gestão em apps/partner-web.

IMPLEMENTAÇÃO E TELAS

Telas: onboarding/documentos e status, visão geral baseada em métricas reais, cadastro da loja, horários/exceções, área e taxas conforme política, cardápio/categorias/adicionais/fotos, disponibilidade, membros/convites/permissões, avaliações, relatórios, extratos/repasses, condições contratuais e suporte. Separar proprietário, gerente, operador e financeiro. Exportações paginadas/assíncronas autorizadas e auditadas. Alteração de conta bancária não liquida dinheiro e requer step-up/aprovação conforme política. Sem dados, não fabricar vendas/gráficos. Portal acessa somente própria organização/unidades.

DADOS, CONTRATOS E ACESSOS

Módulos de loja/catalog/members/reports; financeiro como leitura de M13, não cálculo duplicado.

TESTES OBRIGATÓRIOS

CRUD real, validação upload, escopo por unidade, convite indevido, financeiro negado para operador e exportação cruzada negada.

CRITÉRIOS DE ACEITE

Parceiro consegue cadastrar e manter operação real; nenhuma visão global de outros parceiros.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M07.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M08 — Gestor operacional de pedidos do restaurante

PROMPT M08 — Gestor operacional de pedidos do restaurante

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M07, M10.

OBJETIVO: Construir console de operação dentro do partner-web, separado da gestão.

IMPLEMENTAÇÃO E TELAS

Fila com novo/aceito/em preparo/pronto/retirado/finalizado/cancelado, detalhes e snapshots, alertas sonoros opcionais com consentimento, temporizadores derivados de timestamps reais, aceitar/rejeitar com motivo conforme política, prazo de preparo validado, avisar pronto e confirmar coleta por regra do domínio. Reconexão busca eventos perdidos e evita duplicatas. Conflitos entre operadores tratados por versão/lock otimista. Impressão local explícita; integração impressora/KDS apenas com hardware/API configurados. Pausa da loja e de item reais; pedidos ativos não desaparecem ao pausar. Contato cliente somente necessário e auditável.

DADOS, CONTRATOS E ACESSOS

Commands de pedidos com expected_version/idempotency, subscriptions tenant-scoped, histórico imutável.

TESTES OBRIGATÓRIOS

Aceite simultâneo, transição inválida, reconnect, duplicação de evento, cancelamento concorrente, loja fechada com pedidos ativos e falta de permissão.

CRITÉRIOS DE ACEITE

Pedido real altera estado uma vez; histórico e cliente/entregador recebem atualização autorizada; sem simulação de fila.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M08.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M09 — Portal de redes, franquias e multiunidades

PROMPT M09 — Portal de redes, franquias e multiunidades

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M07, M08, M13.

OBJETIVO: Adicionar gestão consolidada sem romper isolamento.

IMPLEMENTAÇÃO E TELAS

Área no partner-web para organização com múltiplas unidades: seletor de unidade, painel consolidado real, usuários por filial/região, catálogo-base e sobrescritas por loja com regras explícitas, horários locais, relatórios por unidade, exportações e trilha de alterações em lote. Não permitir franchising/compartilhamento entre pessoas jurídicas sem modelo e autorização definidos. Operador de filial não recebe dados globais por erro de filtro. Alterações em lote com prévia, confirmação e resultado parcial por unidade.

DADOS, CONTRATOS E ACESSOS

organization/unit scopes; reports agregados só de unidades autorizadas; operações batch auditadas.

TESTES OBRIGATÓRIOS

Permissões filial/região/rede, consolidação financeira sem duplicar pedidos, operação parcial e cache segregado.

CRITÉRIOS DE ACEITE

Comparativos e gestão em lote só com dados reais e permissão explícita; sem esconder falhas parciais.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M09.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M11 — App do entregador — Android e iOS

PROMPT M11 — App do entregador — Android e iOS

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M03, M10, M14; M13 para ganhos.

OBJETIVO: Construir apps/courier-android e apps/courier-ios com fluxo logístico real.

IMPLEMENTAÇÃO E TELAS

Telas: cadastro/documentos/aprovação, veículo/região, disponibilidade online/offline, oferta com dados comerciais aprovados, aceite/recusa, navegação até loja, chegada e coleta, rota ao cliente, ocorrência, confirmação por código/evidência conforme política, histórico, ganhos/extrato e suporte em rota. Somente após atribuição expor endereço necessário; cliente não deve receber documento/endereço pessoal do entregador. Ofertas expiram e aceite é transacional. Localização em primeiro e segundo plano apenas quando necessário, com consentimento, finalidade, retenção e permissões Android/iOS. Não prometer background com app fechado sem verificar restrições reais de SO. Usar push configurado e estratégia real de reconexão; sem coordenadas simuladas em runtime. Offline registra estado com timestamp e reenvio idempotente apenas quando seguro; não conclui entrega fictícia. Se GPS negado, explicar impacto e suporte, sem criar rastreamento falso.

DADOS, CONTRATOS E ACESSOS

/couriers/me, availability, offers, assignments, locations, delivery commands, earnings; RLS e API só para atribuições permitidas.

TESTES OBRIGATÓRIOS

Aceite simultâneo, oferta expirada, permissão GPS negada, rede instável, timestamp antigo, tentativa de acessar outra rota, código inválido, duplicação de entrega e saldo real.

CRITÉRIOS DE ACEITE

Entrega rastreável com evidências autorizadas e ganhos reconciliados; builds e permissões documentados, pendências de dispositivo explícitas.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M11.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M15 — Portal de operadores logísticos e frotas

PROMPT M15 — Portal de operadores logísticos e frotas

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M11, M14, M13; contratos de frota aprovados.

OBJETIVO: Disponibilizar área restrita de gestão de frota, se modelo contratar frotas.

IMPLEMENTAÇÃO E TELAS

Criar área no admin-web ou portal dedicado somente por ADR, com organização de frota, gestores, entregadores vinculados/autorizados, disponibilidade, entregas delegadas, ocorrências, desempenho e extratos contratuais. Não permitir que gestor de frota leia todos os entregadores independentes nem pedidos de outra frota. Vínculo/saída de entregador com consentimento/documentação e auditoria; ganhos e payout seguem contrato aprovado, não supor vínculo trabalhista ou repasse à frota. Se modelo de frota não aprovado, documentar DESABILITADO, sem telas falsas.

DADOS, CONTRATOS E ACESSOS

fleet organizations/members/contracts e assignments delegadas, respeitando limites de tenant.

TESTES OBRIGATÓRIOS

Frota A versus B, entregador desvinculado, pedido não delegado, extrato e autorização de gestor.

CRITÉRIOS DE ACEITE

Área habilitada só para modelo real contratado; escopo e responsabilidades documentados.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M15.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.

M22 — Extensão mercados, separação e substituições

PROMPT M22 — Extensão mercados, separação e substituições

Aplique integralmente as INSTRUÇÕES PERMANENTES. Leia o código, docs/DELIVERY_STATUS.md e os contratos existentes antes de alterar arquivos. Se este módulo já existir, audite e complete em vez de duplicar.

DEPENDÊNCIAS: M04, M10, M12, M07; só após escopo aprovado.

OBJETIVO: Cobrir supermercado sem distorcer o núcleo inicial.

IMPLEMENTAÇÃO E TELAS

Implementar área de separação no partner-web: lista por pedido, unidade/peso, indisponibilidade, proposta de substituição, consentimento do cliente, atualização de preço e reflexo financeiro. Definir autorização/captura/reembolso compatível com PSP antes de alterar cobrança. Catálogo com unidade de medida e precisão decimal de quantidade, dinheiro em centavos; proibir ajuste arbitrário. Recusa/timeout de substituição seguem política aprovada. Não habilitar medicamento, álcool ou outras categorias reguladas sem revisão específica. Se extensão não aprovada, mantê-la documentada e desabilitada.

DADOS, CONTRATOS E ACESSOS

pick tasks/items, substitution proposals/approvals e ajustes vinculados a pedido/pagamento.

TESTES OBRIGATÓRIOS

Pesagem, arredondamento, substituição recusada/expirada, cobrança insuficiente, reembolso parcial e concorrência.

CRITÉRIOS DE ACEITE

Quantidade/preço final com consentimento e conciliação; sem prometer ajuste PSP não suportado.

ENTREGA DESTA TAREFA

Implemente a fatia funcional possível, execute verificações e atualize documentação do módulo em docs/modules/M22.md. Inclua mapa de rotas/telas, endpoints, permissões, dependências e variáveis por nome (sem valor secreto), testes e runbook. Apresente arquivos e comandos/resultados reais, limitações e próximos passos. Aplicar migrations no Supabase de desenvolvimento autorizado; após testar o código, realizar commit e push no GitHub solicitado, respeitando gates do ambiente; dependência bloqueadora implica status BLOQUEADO com motivo, nunca simulação.